

CODE REVIEW & HEALTH ANALYSIS

FAZAI Code Review & Health Analysis

Comprehensive codebase assessment
of the FAZAI personal cash-basis
accounting PWA application, covering
security, build quality, architecture, and
maintainability.

June 2026

App v0.4.0 | DB v3

Z.a

Table of Contents

1. Executive Summary	2
2. Project Overview	2
3. Critical Findings	3
4. High Priority Findings	6
5. Medium Priority Findings	9
6. Dependency Analysis	12
7. Architecture Assessment	13
8. Recommendations	14

1. Executive Summary

FAZAI is a cash-basis accounting Progressive Web Application (PWA) built on Next.js 16 with Dexie/IndexedDB for client-side data persistence and Zustand for state management. The codebase comprises approximately 75 source files distributed across 7 primary directories, encompassing features such as transaction management, ledger operations, AI-powered financial insights, multi-currency support, and trilingual localization (English, Bahasa Indonesia, Chinese). The application functions well within its minimum viable product (MVP) scope, providing users with a functional personal accounting tool that operates entirely within the browser.

However, this code review and health analysis reveals several critical findings that must be addressed before the application can be considered production-ready. The most severe issues include zero test coverage with no test framework configured, TypeScript build errors deliberately suppressed via `ignoreBuildErrors`, ESLint effectively disabled with over 30 rules set to "off", and more than 15 unused dependencies inflating the bundle. From a security standpoint, PINs are stored in plaintext within IndexedDB (with hardcoded defaults Admin=000000 and User=111111), authentication state resides unencrypted in localStorage, and API routes lack any authentication middleware. Furthermore, despite being marketed as a PWA, the application has no Service Worker, meaning it cannot function offline and fails to meet the core PWA offline-first requirement. Significant code duplication exists between settings and admin-settings components, and N+1 query patterns in the admin accounts view will degrade performance as datasets grow. These findings collectively indicate that while FAZAI demonstrates competent feature delivery for its MVP scope, it carries critical security vulnerabilities and maintainability gaps that pose substantial risk in a production environment.

2. Project Overview

Attribute	Value
Application	FAZAI — Personal Cash-Basis Accounting PWA
Framework	Next.js 16.1.3 (Turbopack) + React 19
Database	Dexie 4.4.3 (IndexedDB, client-side)
State Management	Zustand 5.0.6
UI Library	Radix UI + shadcn/ui + Tailwind CSS
AI Integration	z-ai-web-dev-sdk 0.0.17
Export	jsPDF + xlsx

Auth	6-digit PIN (client-side, plaintext)
Languages	English, Bahasa Indonesia, Chinese
App Version	0.4.0
DB Schema Version	3
Source Files	~75 across 7 directories
Test Coverage	0%

Table 1: FAZAI Project Metadata

3. Critical Findings

The following findings are classified as **Critical** severity. These issues represent immediate risks to security, data integrity, or application reliability and must be addressed before any production deployment.

#	Category	Finding	Impact
1	Security	PINs stored in plaintext in IndexedDB (Admin=000000, User=111111)	Data breach if browser storage accessed
2	Security	Auth state stored unencrypted in localStorage	Session hijacking via XSS
3	Security	No authentication on API routes (/api/ai/*)	Unauthorized API access
4	Build	ignoreBuildErrors: true in next.config.ts masks all TypeScript errors	Type safety violations go undetected
5	Build	ESLint effectively disabled (30+ rules set to "off")	Code quality issues unreported
6	PWA	No Service Worker — PWA cannot work offline	App breaks without network on initial load
7	Quality	Zero test coverage — no test framework configured	No regression protection

Table 2: Critical Severity Findings

3.1 Plaintext PIN Storage

The application stores user PINs in plaintext within IndexedDB, with hardcoded default values of Admin=000000 and User=111111. This is a fundamental security vulnerability that violates even the most basic authentication security principles. Any code running in the browser context, including malicious browser extensions, cross-site scripting payloads, or debugging tools, can trivially read these PINs from IndexedDB. Furthermore, the hardcoded default PINs are easily guessable, and there is no mechanism to force users to change them on first login. The combination of plaintext storage and weak default credentials means that an attacker with any form of browser access can bypass authentication entirely. Proper remediation requires hashing PINs using bcrypt or argon2 with per-user salts, enforcing PIN changes from defaults, and implementing account lockout policies after failed attempts.

3.2 Unencrypted Auth State in localStorage

Authentication state is stored unencrypted in the browser's localStorage, making it accessible to any JavaScript code running in the same origin. This creates a direct path for session hijacking through cross-site scripting (XSS) attacks. If an attacker can inject malicious JavaScript into the application through any of its input fields, AI chat interface, or third-party dependencies, they can read the authentication tokens from localStorage and impersonate any user. Unlike httpOnly cookies, localStorage data is always accessible to client-side JavaScript, providing no isolation barrier. The impact is compounded by the fact that the application has no Content Security Policy headers and no XSS sanitization framework. Remediation should involve migrating to secure, httpOnly, SameSite cookies for session management, implementing proper CSRF tokens, and adding Content Security Policy headers to restrict script execution contexts.

3.3 No API Route Authentication

All API routes under /api/ai/* accept requests without any authentication or authorization checks. Any client that can reach the server can invoke these endpoints, potentially consuming AI API credits, extracting financial data, or performing unauthorized operations. The AI chat endpoint is particularly concerning because it processes user financial data and sends it to external AI services. Without authentication, an attacker could automate requests to these endpoints, leading to financial loss through API credit consumption and potential data exfiltration. The routes also lack rate limiting, meaning an attacker could flood the AI endpoints with requests. Proper remediation requires adding authentication middleware to all API routes, implementing rate limiting, and validating all incoming request payloads against defined schemas.

3.4 TypeScript Build Errors Suppressed

The next.config.ts file sets ignoreBuildErrors: true, which causes the TypeScript compiler to skip type checking during the build process. This means that any type errors, incorrect type annotations, missing properties, and type mismatches are silently ignored rather than caught at build time. TypeScript's

primary value proposition is catching bugs through static type analysis, and disabling this check entirely negates that benefit. The codebase likely contains numerous type violations that would be caught by a properly configured build. Additionally, non-null assertions (!) are used extensively on `Array.find()` results, which can cause runtime errors when the array lookup returns undefined. The combination of suppressed build errors and liberal use of non-null assertions creates a false sense of type safety while the code actually operates with significant type unsoundness. Remediation requires removing `ignoreBuildErrors`, fixing all TypeScript errors that surface, and replacing non-null assertions with proper null checks.

3.5 ESLint Effectively Disabled

The ESLint configuration has over 30 rules explicitly set to "off", including critical rules for catching unused variables, unreachable code, missing return statements, and improper hook usage. This effectively disables the linter's ability to catch common programming mistakes and enforce code quality standards. Without active linting, developers receive no automated feedback on code quality issues, leading to an accumulation of technical debt. Dead code, unused imports, and inconsistent patterns can proliferate unchecked. The disabled rules include many that are enabled by default in recommended configurations, suggesting the rules were turned off to suppress existing warnings rather than fixing the underlying issues. Progressive remediation should start by enabling the most critical rules (unused variables, unreachable code, React hooks rules), fixing the violations, and then gradually enabling additional rules in subsequent iterations.

3.6 No Service Worker for PWA

Despite having a web app manifest that qualifies FAZAI as a Progressive Web Application, the application has no registered Service Worker. This is a fundamental PWA requirement, as the Service Worker enables offline functionality, background synchronization, and push notifications. Without it, the application cannot function without an active network connection, which defeats the core value proposition of a PWA. On initial load, if the network is unavailable, the application simply fails to load. For an accounting application where users may need to access financial data in various network conditions, this is particularly problematic. The manifest file exists with proper configuration, but without a Service Worker to intercept network requests and serve cached content, the manifest alone provides no offline capability. Implementing a Service Worker with appropriate caching strategies (cache-first for static assets, network-first for API calls) would enable true offline functionality and bring the application closer to meeting PWA standards.

3.7 Zero Test Coverage

The application has zero test coverage, with no test framework configured in the project. There are no unit tests, integration tests, or end-to-end tests of any kind. This means that every code change carries

the risk of introducing regressions that will go undetected until they manifest as user-facing bugs. The core business logic in ledger-engine.ts, which handles financial calculations, account balancing, and transaction processing, has no automated verification. Given the critical nature of financial calculations, even small rounding errors or logic mistakes could lead to incorrect financial reporting. The absence of tests also makes refactoring extremely risky, as developers cannot verify that behavior is preserved after code changes. Setting up a test framework (Vitest is recommended for its Vite compatibility and speed) and achieving at minimum 50% coverage on core financial logic should be a top priority.

4. High Priority Findings

The following findings are classified as **High** priority. These issues pose significant risks to application performance, maintainability, or security but do not represent immediate production-blocking vulnerabilities.

#	Category	Finding	Impact
1	Dependencies	15+ unused packages (next-auth, prisma, react-query, react-table, recharts, etc.)	Bundle bloat, maintenance burden
2	Duplication	settings.tsx nearly identical to admin-settings.tsx	Maintenance risk
3	Performance	N+1 queries in admin-accounts.tsx; full table scans in dashboard	Slow rendering with large datasets
4	Reliability	No React Error Boundaries	Any render error crashes entire app
5	Security	No rate limiting on PIN login attempts	Brute force vulnerability
6	Security	AI endpoint trusts client-sent account IDs without validation	Data integrity risk
7	Dead Code	Prisma + next-auth + react-query + react-table installed but never used	~1MB+ wasted bundle

Table 3: High Priority Findings

4.1 Unused Dependencies (15+ Packages)

The project has more than 15 packages installed that are never imported or used anywhere in the codebase. These include substantial libraries such as next-auth (~200KB), the entire Prisma ORM with

its client (~15MB combined), @tanstack/react-query (~40KB), @tanstack/react-table (~60KB), @dnd-kit/core + sortable (~30KB), @mdxeditor/editor (~500KB+), react-markdown (~40KB), react-syntax-highlighter (~200KB+), sharp (~10MB), recharts (~200KB), and react-hook-form (~30KB). Beyond the direct bundle size impact, unused dependencies increase install time, expand the security attack surface, create confusion for new developers about which technologies are actually in use, and may introduce breaking changes during updates. The sharp package alone adds approximately 10MB to node_modules and is particularly problematic as it requires native binaries. A thorough audit and removal of all unused packages would significantly reduce bundle size, improve build times, and reduce the security maintenance burden.

4.2 Code Duplication: settings vs admin-settings

The settings.tsx and admin-settings.tsx components contain nearly identical code, with only minor differences in the available options and permission checks. This duplication means that any bug fix or feature addition must be applied to both files independently, doubling the maintenance effort and increasing the risk of divergence where one file gets updated while the other does not. The duplicated code includes the entire settings UI layout, form handling logic, and state management. A proper remediation would extract the shared logic into a reusable SettingsPanel component that accepts configuration props for the differences between admin and user settings, with each page rendering the shared component with appropriate parameters. This pattern would also make it easier to add new setting types or modify the existing layout without the risk of inconsistent behavior.

4.3 Performance: N+1 Queries and Full Table Scans

The admin-accounts.tsx component exhibits N+1 query patterns where it first retrieves a list of accounts and then makes individual queries for each account's related data within a loop. This pattern causes the number of database queries to scale linearly with the number of accounts, leading to severe performance degradation as the dataset grows. Similarly, the dashboard component performs full table scans of the transactions table to compute summary statistics, without leveraging any database indexes or pre-computed aggregations. For a financial application where datasets can grow significantly over time, these patterns will result in increasingly slow page loads. Dexie supports compound indexes and bulk queries that can eliminate these issues. Adding a compound index on [accountId+year+month] for monthly summaries, using Dexie's where().equals() for targeted lookups instead of toArray() filters, and implementing cursor-based pagination would address these performance concerns.

4.4 No React Error Boundaries

The application has no React Error Boundaries implemented anywhere in its component tree. Without Error Boundaries, any unhandled JavaScript error during rendering, in a lifecycle method, or in a constructor of any child component will cause the entire application to crash and display a white screen.

There is no graceful degradation or user-friendly error messaging. For a financial application where users may be in the middle of entering transaction data, a crash could result in data loss. React Error Boundaries provide a way to catch rendering errors and display fallback UI while keeping the rest of the application functional. At minimum, Error Boundaries should be implemented at the route level (around each page component) and around critical interactive components like the transaction form and AI chat interface. Additionally, error reporting integration should be considered to capture and track runtime errors in production.

4.5 No Rate Limiting on PIN Login

The PIN login mechanism has no rate limiting or account lockout policy. An attacker can attempt unlimited PIN combinations without any delay or lockout, making brute force attacks trivial. With only 6-digit PINs (1,000,000 possible combinations), a simple script could enumerate all possibilities in minutes. The default PINs of 000000 and 111111 would be among the first attempted in any brute force attack. Proper remediation includes implementing exponential backoff on failed attempts (e.g., 1s, 2s, 4s, 8s...), account lockout after a configurable number of failed attempts (typically 5), requiring CAPTCHA after multiple failures, and logging all authentication attempts for audit purposes. Since this is a client-side application, server-side rate limiting is not directly applicable, but client-side throttling combined with IndexedDB-based attempt tracking would provide meaningful protection.

4.6 AI Endpoint Trusts Client-Sent Account IDs

The AI chat endpoint accepts account IDs from the client without server-side validation that the requesting user has access to those accounts. This means a malicious client can send arbitrary account IDs and potentially access financial data belonging to other users or perform operations on accounts they should not have access to. While the current architecture is primarily single-user and client-side, this represents a significant data integrity risk if the application ever moves to a multi-user or server-side data model. The endpoint also does not validate that the account IDs conform to expected formats, opening the door to injection attacks or unexpected behavior with malformed inputs. All client-sent identifiers must be validated server-side, and the server must independently verify that the requesting session has authorization to access the specified resources.

4.7 Dead Code: Major Unused Packages

Several major packages are installed but never used in the codebase. Prisma ORM and its client add approximately 15MB to the project, with a schema that appears to be a generic template unrelated to the application's data model. Next-auth is installed but the application uses a custom PIN-based authentication system instead. React Query and React Table are installed but neither is imported in any component. These packages collectively account for well over 1MB of wasted bundle space and significantly increase the project's dependency footprint. Beyond size, each unused dependency

represents an ongoing maintenance burden: they still receive updates that must be reviewed, they appear in security audit reports requiring investigation, and they may have peer dependency conflicts with packages that are actually used. A clean removal of all unused packages should be performed, followed by verification that the application builds and runs correctly without them.

5. Medium Priority Findings

The following findings are classified as **Medium** priority. These issues affect code quality, maintainability, or user experience but do not pose immediate security or reliability risks.

#	Category	Finding	File(s)
1	i18n	15+ inline ternaries instead of translation keys	Multiple components
2	Type Safety	Non-null assertions on Array.find() results	transaction-form.tsx
3	Type Safety	any types in API routes and report viewer	ai-chat/route.ts, report-viewer.tsx
4	Performance	No memoization on expensive computed values	report-viewer.tsx
5	Memory	setTimeout without cleanup in pin-login.tsx	pin-login.tsx
6	Accessibility	Click divs without role/tabIndex, missing aria labels	dashboard, history, ai-chat
7	Duplication	Date helpers duplicated between format.ts and ledger-engine.ts	format.ts, ledger-engine.ts
8	UX	TOAST_REMOVE_DELAY = 1,000,000ms (16+ minutes)	use-toast.ts

Table 4: Medium Priority Findings

5.1 Inline Ternaries Instead of Translation Keys

The application implements internationalization (i18n) with a custom translation system, but over 15 instances use inline ternary operators (e.g., `lang === "en" ? "Income" : "Pendapatan"`) instead of referencing translation keys through the i18n system. This pattern is fragile and error-prone: any change to the displayed text requires finding and updating every inline ternary rather than modifying a single

translation key. It also makes it difficult to add new languages, as developers must search the entire codebase for inline ternaries and add additional conditions. The proper approach is to use the existing translation system consistently, defining all user-facing strings in translation files and referencing them via translation keys in components. This centralizes all translatable text and makes the codebase more maintainable and extensible.

5.2 Non-null Assertions on Array.find()

The `transaction-form.tsx` file uses non-null assertion operators (!) on `Array.find()` results. The `Array.find()` method returns undefined when no matching element is found, but the non-null assertion tells TypeScript to treat the result as always defined. This creates a type safety hole that can lead to runtime errors when the array lookup legitimately returns undefined, which can occur when data is in an inconsistent state or when the referenced entity has been deleted. Instead of using non-null assertions, the code should handle the undefined case explicitly, either by providing a default value, showing an error state, or using optional chaining. This is especially important in a financial application where accessing properties on undefined objects could cause calculation errors or crashes.

5.3 any Types in API Routes and Report Viewer

The AI chat route (`ai-chat/route.ts`) and report viewer component (`report-viewer.tsx`) use TypeScript's `any` type, which completely bypasses type checking. In the API route, this means request and response objects are not validated against any schema, allowing malformed data to flow through the system undetected. In the report viewer, `any`-typed props and state variables obscure the expected data shapes, making it impossible for TypeScript to catch type mismatches during development. The `any` type is considered harmful because it provides zero type safety while giving a false impression that the code is typed. Proper remediation involves defining explicit interfaces for all data shapes, using Zod or similar libraries for runtime schema validation in API routes, and replacing `any` with specific types or at minimum with `unknown` (which requires explicit type narrowing before use).

5.4 No Memoization on Expensive Computed Values

The `report-viewer.tsx` component performs expensive computations (filtering, sorting, and aggregating financial data) on every render without any memoization. These computations involve iterating over the entire transaction dataset multiple times to generate various report views. Without `useMemo` or similar memoization, these calculations are recalculated on every render cycle, even when the underlying data has not changed. As the transaction dataset grows, this will cause increasingly noticeable lag during user interactions. Implementing `useMemo` for derived data computations and `useCallback` for event handlers passed as props would prevent unnecessary recalculations and significantly improve rendering performance, particularly for users with large transaction histories.

5.5 setTimeout Without Cleanup in pin-login.tsx

The pin-login.tsx component uses setTimeout for delayed actions (such as navigation after successful login) without cleaning up the timeout in a useEffect cleanup function. If the component unmounts before the timeout fires (for example, if the user navigates away or the component re-renders), the timeout callback will still execute, potentially causing state updates on an unmounted component. This can lead to React warnings about memory leaks and, in more severe cases, cause unexpected behavior such as navigation events firing at inappropriate times. The fix is straightforward: store the timeout ID and clear it in the useEffect cleanup function. This is a common React pattern that ensures no side effects persist after component unmounting.

5.6 Accessibility Issues: Missing Roles and ARIA Labels

Several interactive elements across the dashboard, history, and AI chat components are implemented as clickable div elements without proper ARIA roles, tabIndex attributes, or aria-label descriptions. This makes the application completely inaccessible to screen reader users and keyboard-only navigation. Interactive elements must have appropriate ARIA roles (button, link, tab, etc.), tabIndex for keyboard focus, and descriptive aria-label attributes. For a financial application where users with disabilities may need to manage their finances, accessibility is not merely a nice-to-have but a legal requirement in many jurisdictions. A comprehensive accessibility audit using tools like axe-core or Lighthouse, followed by systematic remediation of all identified issues, would bring the application closer to WCAG 2.1 AA compliance.

5.7 Duplicated Date Helper Functions

Date formatting and manipulation helper functions are duplicated between format.ts and ledger-engine.ts. Both files contain similar logic for date parsing, formatting, and month/year extraction, but with subtle differences in implementation that could lead to inconsistent behavior. This duplication violates the DRY (Don't Repeat Yourself) principle and increases the maintenance burden, as any bug fix or feature addition to date handling must be applied in both locations. The proper solution is to consolidate all date-related utilities into a single module (format.ts is the natural location) and import them where needed. This ensures consistent behavior across the application and reduces the surface area for date-related bugs, which are particularly impactful in a financial application where date-based calculations determine monthly summaries and fiscal period boundaries.

5.8 Excessive Toast Removal Delay

The toast notification system in use-toast.ts has a TOAST_REMOVE_DELAY constant set to 1,000,000 milliseconds (approximately 16.67 minutes). This means that toast notifications remain in the DOM and application state for an extraordinarily long time after they are visually dismissed. While this does not

directly cause user-visible bugs, it creates a memory leak where toast state accumulates over the course of a session, particularly if the application generates many toast notifications during normal use (such as during bulk transaction imports or repeated form submissions). The excessive delay also means that the internal toast management system retains references to notification data long after it is relevant. A more reasonable value would be 1,000-5,000ms (1-5 seconds), which provides sufficient time for any exit animations to complete while promptly cleaning up stale state.

6. Dependency Analysis

The following table identifies packages installed in the project that are never imported or used in the codebase. Removing these unused dependencies would significantly reduce bundle size, improve build performance, and decrease the security maintenance burden.

Package	Estimated Size	Status
next-auth	~200KB	Installed, never imported
next-intl	~50KB	Installed, custom i18n used
prisma + @prisma/client	~15MB	Schema is generic template, never used
@tanstack/react-query	~40KB	Never imported
@tanstack/react-table	~60KB	Never imported
@dnd-kit/core + sortable	~30KB	Never imported
@mdxeditor/editor	~500KB+	Never imported
react-markdown	~40KB	Never imported
react-syntax-highlighter	~200KB+	Never imported
sharp	~10MB	Never imported
recharts	~200KB	Never imported
react-hook-form	~30KB	Never imported

Table 5: Unused Dependencies

Security Advisory: The xlsx package (version 0.18.5) has a known vulnerability, CVE-2024-22400, which is a prototype pollution issue. This vulnerability could allow an attacker to modify the prototype of base objects, potentially leading to privilege escalation or denial of service. Given that xlsx is actively

used in the application for financial data export, this represents a tangible security risk. The recommended remediation is to replace `xlsx` with a more secure alternative such as `exceljs` or `xlsx-populate`, both of which do not have known prototype pollution vulnerabilities and provide comparable functionality for spreadsheet generation. If replacement is not immediately feasible, the `xlsx` package should be updated to the latest version and its usage should be audited to ensure that untrusted data is not passed to vulnerable functions.

7. Architecture Assessment

Data Flow

FAZAI employs a purely client-side architecture where all application data resides in the browser's IndexedDB via `Dexie.js`. There is no server-side database or API for data persistence. The application uses `Zustand` for in-memory state management, with changes persisted to IndexedDB through `Dexie`'s reactive observation system. Notably, the application implements custom single-page application (SPA) routing that bypasses `Next.js`'s App Router, meaning the framework's server-side rendering and routing capabilities are largely unused. Every component in the application is marked with the `'use client'` directive, confirming that the entire rendering pipeline operates on the client side. This architecture simplifies deployment and eliminates server costs, but it creates fundamental limitations around data portability, multi-device synchronization, and server-side security enforcement.

Database Design

The database layer uses `Dexie` version 3 with a well-structured schema that includes a compound index on `[accountId+year+month]` for efficient monthly summary queries. The schema migration path from version 1 through version 3 is properly handled with incremental upgrade functions. However, the `archivedTransactions` table lacks a date index, which will cause performance issues when querying archived records by date range. The current schema supports the application's cash-basis accounting model but would need significant extension to support accrual accounting or multi-entity structures. The reliance on IndexedDB also means that data is tied to a specific browser profile on a specific device, with no built-in mechanism for data export/import across devices beyond the manual export functionality.

API Pattern

The API layer is extremely thin, consisting only of AI-related routes under `/api/ai/*`. These routes follow an unusual pattern where all financial data is sent from the client to the server in each request, creating a data round-trip that negates many of the benefits of a server-side API. There is no server-side data persistence, no caching of AI responses, and no authentication middleware. The API routes essentially

serve as thin proxies between the client and the external AI service, with the client bearing full responsibility for data preparation and response handling. This pattern is fundamentally a client-side application with a minimal API layer for AI integration, rather than a traditional client-server architecture. While this simplifies the backend, it means that all security and data integrity checks must be implemented on the client, where they can be bypassed.

Security Model

The application's security model is based on a 6-digit PIN with plaintext storage, no account lockout mechanism, and no session management. The next-auth package is installed but completely unused, suggesting it was initially considered but replaced with the simpler PIN approach. There is no encryption of data at rest, no secure session token management, and no CSRF protection. The authentication system provides only the appearance of security without meaningful protection against determined attackers. Any code running in the same browser context can bypass authentication by directly reading and writing to IndexedDB and localStorage. A production-grade security model would require hashing PINs, implementing session tokens with expiration, adding CSRF protection, and ideally moving authentication verification to a server-side component that the client cannot bypass.

PWA Status

The application has a valid web app manifest with proper icons, theme colors, and display configuration. However, the absence of a Service Worker means the application cannot meet the core PWA requirement of offline functionality. Without a Service Worker, the browser cannot cache the application shell or intercept network requests to serve cached content when offline. The application will fail to load on initial visit without network connectivity, and even cached pages will not be able to load dynamic content. Implementing a Service Worker with appropriate caching strategies (cache-first for static assets, stale-while-revalidate for API responses, and cache-only for the application shell) would enable true offline functionality and qualify the application for PWA installation prompts across browsers.

8. Recommendations

8.1 Immediate Actions (Week 1)

The most urgent actions address critical security vulnerabilities and build quality issues that currently mask underlying problems. First and foremost, PIN storage must be migrated from plaintext to hashed storage using bcrypt or argon2 with per-user salts. This change should include forcing users to change default PINs on first login. Second, authentication middleware must be added to all API routes under /api/ai/, verifying that requests include valid session tokens before processing. Third, the

ignoreBuildErrors setting must be removed from next.config.ts, and all resulting TypeScript errors must be fixed. While this may surface a significant number of errors, each one represents a potential runtime bug that is currently hidden. Fourth, ESLint rules should be progressively re-enabled, starting with the most critical rules: no-unused-vars, no-unreachable, react-hooks/rules-of-hooks, and react-hooks/exhaustive-deps. Fixing the violations for these rules will immediately improve code quality and catch common mistakes. These four actions, if completed within the first week, will dramatically improve the application's security posture and developer experience by restoring the type safety and code quality tooling that is currently disabled.

8.2 Short-term (Month 1)

Within the first month, the focus should shift to removing technical debt and implementing reliability improvements. The 15 unused dependencies should be audited and removed, starting with the largest packages (Prisma at ~15MB, sharp at ~10MB, @mdxeditor/editor at ~500KB+). Removing these packages will reduce the project's node_modules by approximately 25MB and eliminate corresponding security audit noise. React Error Boundaries should be implemented at the route level and around critical components (transaction form, AI chat, report viewer) to prevent full application crashes from component-level errors. A Service Worker should be implemented using Workbox or a similar library to enable offline caching of the application shell and static assets. Client-side rate limiting should be added to the PIN login mechanism, with exponential backoff and account lockout after five failed attempts. Finally, the duplicated settings and admin-settings components should be refactored into a shared SettingsPanel component with configuration props, eliminating the current maintenance risk of parallel code paths.

8.3 Medium-term (Quarter 1)

The first quarter should focus on establishing testing infrastructure and improving security and reliability. Vitest should be configured as the test framework, along with Testing Library for React component testing. The initial testing target should be 50% coverage on ledger-engine.ts, which contains the core financial calculation logic that is most critical to verify. The xlsx package (version 0.18.5) should be replaced with a secure alternative such as exceljs, eliminating the CVE-2024-22400 prototype pollution vulnerability. Proper authentication with session tokens should be implemented, moving away from the current PIN-only system to a more robust session management approach with token expiration and refresh mechanisms. Input validation should be added to all API routes using a schema validation library like Zod, ensuring that all incoming request data conforms to expected shapes before processing. Additionally, the any types in API routes and the report viewer should be replaced with properly defined TypeScript interfaces, and non-null assertions on Array.find() results should be replaced with explicit null checks.

8.4 Long-term

Long-term improvements should focus on architectural evolution that enables the application to scale beyond its current single-device, single-user limitations. The most significant change would be migrating to server-side persistence, where financial data is stored in a proper database (PostgreSQL or similar) rather than client-side IndexedDB. This migration would enable cross-device synchronization, allowing users to access their financial data from any device with proper authentication. End-to-end testing should be implemented using Playwright, covering critical user flows such as transaction creation, account management, and report generation. Role-based access control (RBAC) should be implemented with proper session management, enabling multi-user scenarios where different users have different permission levels. The current PIN-based system would be replaced with a proper authentication service supporting email/password, OAuth providers, and multi-factor authentication. Data encryption at rest should be implemented for sensitive financial data, and a comprehensive audit logging system should track all data modifications for compliance and forensic purposes. These long-term improvements would transform FAZAI from a single-user client-side tool into a robust, multi-user financial management platform.